PROJECT PROPOSAL & TECHNICAL SPECIFICATION

IMPOSTOR: Local Peer-to-Peer Social Deduction Game

Zero-Cloud Offline Multiplayer Engine with Embedded CPython & Custom DSA Architecture

Project Category: Distributed Mobile Systems & Custom DSA	**Target Platform:** Android (Mobile) & Cross-Platform (Flutter)
Core Stack: Flutter / Dart, Python 3.12 (CPython), WebSockets	**Team Composition:** 3 Dedicated Engineering Team Members
Networking Model: Local Wi-Fi / Hotspot Peer-to-Peer (Zero-Cloud)	**Document Version:** 1.0 (Full Architecture, Roadmap & Contribution Roles)

1. Executive Summary & Problem Statement

Executive Summary: *Impostor* is an offline-first social deduction mobile game engineered to deliver instant, high-fidelity party gaming completely independent of cloud servers, external databases, or internet connectivity. By packaging an entire asynchronous Python 3.12 game engine directly inside a Flutter mobile application, any player can create a game room over a local Wi-Fi router or portable mobile hotspot, allowing nearby players to connect peer-to-peer via fast camera QR scanning or manual IP entry with microsecond responsiveness.

Problem Statement: Commercial multiplayer party titles (such as *Among Us* or *Jackbox*) universally mandate external cloud matchmaking and persistent internet connections. This creates four severe failure modes: (1) *Zero Offline Capability:* Games are unplayable during flights, road trips, outdoor gatherings, or in areas with captive portals; (2) *Server Deprecation & Outages:* Titles become unplayable whenever central servers experience outages or developer sunsets; (3) *Privacy Hazards:* User interactions and device metadata are routed through third-party telemetry servers; and (4) *Packet Latency:* Unnecessary round-trips to distant cloud servers impair live, real-time deduction.

Proposed Solution: *Impostor* decentralizes the game loop entirely. The host phone runs both the Flutter UI and a local background Python WebSocket server. Communication occurs exclusively within the local subnet, delivering 100% data privacy, zero recurring cloud hosting costs, and resilience across any environment.

2. Core Technical Innovations & Mobile Systems Engineering

A. In-Process Python Supervisor Daemon (Loopback Control Channel)
Embedded CPython inside Android cannot safely re-initialize once loaded (Py_Initialize() is a process singleton). To permit starting and stopping game rooms without restarting the Flutter app, a dedicated supervisor daemon (server/embedded_entry.py) runs on an asyncio loop. It exposes a loopback-only control channel strictly bound to 127.0.0.1:8766, authenticated with a dynamic 256-bit token. Commands (START, STOP, STATUS, QUIT) cleanly instantiate and tear down the game server, releasing port 8765 completely on demand.

B. Zero-Configuration Local Discovery & Hotspot Fallback Subsystem
Standard mobile IP lookups fail when the host enables a Portable Hotspot instead of connecting to Wi-Fi. *Impostor* implements a multi-tiered network resolver that enumerates active network interfaces (wlan, ap, softap) and filters RFC 1918 private subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16). The host lobby encodes the endpoint into an instant QR payload (QrPayload) for camera auto-joining.

C. Android 14+ Foreground Service Integration (specialUse)
To prevent Android OEM battery management from killing the server socket when the host screen dims, the application integrates flutter_foreground_task with the Android 14 specialUse foreground service subtype and partial CPU wake locks.

3. Custom Data Structures & Algorithmic Foundations (Built from Scratch)

A central requirement of the project is that all scheduling, indexing, and state pipeline algorithms are implemented from first principles in pure Python, bypassing standard library conveniences (e.g., collections.deque, heapq, or default dictionary room lookups):

Data Structure	Engine Implementation Role	Complexity	Algorithmic Advantage & Design Rationale
Circular Doubly-Linked List **CircularLinkedList[T]**	Manages player clue turn rotation and cyclic round progression.	**Step:** O(1) **Append:** O(1) **Remove:** O(1)	Enables infinite circular iteration over active players without modulo indexing; supports seamless O(1) removal of disconnected players mid-round.
Binary Min-Heap **MinHeap[T]**	Priority timer scheduler for discussion countdowns and vote deadlines.	**Peek:** O(1) **Push:** O(log N) **Pop:** O(log N)	Enables asynchronous timer management where the 100ms ticker only checks the heap root rather than iterating through all rooms and events.
FIFO Queue **Queue[T]**	Serializes incoming player actions, clue submissions, and votes.	**Enqueue:** O(1) **Dequeue:** O(1) **Peek:** O(1)	Guarantees deterministic, sequential event handling and race-condition elimination during concurrent high-frequency vote submissions.
Chained Hash Table **ChainedHashTable[K,V]**	Core room registry managing active rooms, codes, and player lookups.	**Average:** O(1) **Worst:** O(N) **Resize:** O(N)	Custom djb2 polynomial hash with separate collision chaining; dynamically doubles capacity at 0.75 load factor to ensure constant-time retrieval.

4. Team Structure & Contribution Roles (3 Members)

To achieve balanced workload distribution, modular architectural separation, and clear engineering accountability, the project is split across three dedicated domain leads:

MEMBER 1: Core Engine & Algorithmic Logic Lead | *Stack: Python 3.12, Pytest, Custom Object-Oriented DSA*

- **Custom DSA Engine:** Architect and code all 4 foundational data structures from scratch (CircularLinkedList, MinHeap, Queue, ChainedHashTable with djb2 hashing), including custom iterators and dynamic resizing.
- **Game State Machine:** Construct the complete room lifecycle finite state machine (Lobby, RoleReveal, CluePhase, Discussion, Voting, Results) in engine/game.py.
- **Word Bank & Validation:** Build the word repository (engine/words.py) supporting 120+ curated words across 10 categories, semantic clue validation rules, and secret word matching algorithms.
- **Asymmetric Information Secrecy:** Enforce mathematical client-view filtering (crew views never reveal Impostor identity during active play; impostor views never reveal the Secret Word prior to round resolution).
- **Verification:** Author comprehensive Pytest suites (test_structures.py, test_engine_flow.py, test_protocol.py).

MEMBER 2: Embedded Systems & Server Architecture Lead | *Stack: Python asyncio, websockets, serious_python, Bash*

- **In-Process Supervisor:** Design the long-lived supervisor daemon (server/embedded_entry.py) running an asyncio event loop, overcoming CPython process singleton constraints on mobile OSs.
- **Loopback Control Protocol:** Implement the authenticated loopback control channel on 127.0.0.1:8766 with random 256-bit token authorization, processing START, STOP, STATUS, and QUIT commands safely.
- **Teardown & Port Release:** Engineer graceful shutdown routines in server/main.py and server/connection_manager.py (broadcasting room_closed, closing sockets, canceling ticker tasks, and fully releasing port 8765).
- **Automated Android Packaging:** Create the packaging pipeline (scripts/package_python.sh) to bundle pure-Python wheels (websockets==17.3) and engine files into compact 32KB assets for Android ABIs (arm64, armeabi, x86_64).
- **Verification:** Author supervisor lifecycle and concurrency tests (tests/test_supervisor.py, tests/test_server_integration.py).

MEMBER 3: Mobile Client, UI/UX & Native Android Lead | *Stack: Dart, Flutter Framework, Android NDK, Kotlin/Gradle*

• **Flutter UI/UX Architecture:** Design and implement the complete suite of reactive client screens (HomeScreen, LobbyScreen, RoleRevealScreen, ClueSubmissionScreen, VotingScreen, ResultsScreen).
• **Host Controller State Machine:** Develop HostService and HostControlClient in Dart, managing supervisor launch, token generation, TCP control channel communication, socket polling, and emergency process exit fallbacks.
• **Camera QR Scanning & Generation:** Integrate mobile_scanner with custom viewfinder overlays and qr_flutter rendering for zero-friction peer-to-peer room pairing.
• **Android Native Services:** Configure Android 14 specialUse Foreground Service notifications and CPU wake locks (wakelock_plus) to ensure uninterrupted hosting when the host device is locked or backgrounded.
• **Network Discovery & Debug UI:** Build NetworkIpHelper for Wi-Fi and hotspot RFC 1918 IP resolution, and implement the EmbeddedServerDebugScreen providing live telemetry and manual server control.

5. RACI Responsibility & Accountability Assignment Matrix

Definition of terms: **R** = Responsible (performs work), **A** = Accountable (final decision/ownership), **C** = Consulted (provides input), **I** = Informed (kept updated of progress).

Deliverable / Workstream	Member 1 (Engine)	Member 2 (Systems)	Member 3 (Client)
Custom DSA Core (Linked List, Heap, Queue, Hash Table)	**Responsible / Accountable**	Consulted (API Interface)	Informed
Game Rules Finite State Machine & Secrecy Filters	**Responsible / Accountable**	Consulted (Async hooks)	Consulted (UI States)
Word Repository & Semantic Clue Matcher	**Responsible / Accountable**	Informed	Consulted (Language UI)
In-Process Supervisor & Loopback Control Channel	Informed	**Responsible / Accountable**	Consulted (Socket Spec)
WebSocket Server, Ticker Loop & Socket Teardown	Consulted (Engine Calls)	**Responsible / Accountable**	Consulted (Disconnect UI)
Python ABI Bundling & Packaging Script	Informed	**Responsible / Accountable**	Consulted (Assets)
Flutter Client Screens, State & View Models	Consulted (State Display)	Informed	**Responsible / Accountable**
HostService Daemon Manager & Port Polling	Informed	Consulted (Control Commands)	**Responsible / Accountable**
Android Foreground Service & CPU WakeLocks	Informed	Consulted (Thread Safety)	**Responsible / Accountable**
Camera QR Scanning & Network Interface Parser	Informed	Informed	**Responsible / Accountable**
Comprehensive Testing, Profiling & Documentation	Responsible (Pytest Suite)	Responsible (Supervisor)	Responsible (Flutter/UI)

6. Project Implementation Timeline & Milestone Deliverables

Phase / Milestone	Key Technical Objectives & Outputs	Duration	Primary Lead
Phase 1: DSA & Engine Core	Design 4 custom data structures, word repository, and game state machine engine.	Weeks 1 - 2	Member 1
Phase 2: Networking & Protocol	Asyncio WebSocket server, connection manager, client heartbeats, and CLI test client.	Weeks 3 - 4	Member 2
Phase 3: Flutter UI & QR	Full reactive Flutter UI (Lobby, Reveal, Clue, Voting, Results) and camera QR scanner.	Weeks 5 - 6	Member 3
Phase 4: Embedded CPython	Bundle CPython 3.12 with serious_python, package pure wheels, and test Android builds.	Weeks 7 - 8	Member 2 & 3
Phase 5: Supervisor & Control	Loopback control channel on 127.0.0.1:8766, clean socket teardown fix, and HostService.	Weeks 9 - 10	Member 2 & 3
Phase 6: Hardening & Testing	Comprehensive testing (54 pytest, 28 flutter), zero lint issues, and multi-phone trials.	Weeks 11 - 12	All Members

7. Risk Analysis & Engineering Mitigation Strategies

Identified Technical Risk	Impact	Engineered Architectural Mitigation Strategy
CPython Process Singleton **CPython cannot be re-initialized once loaded into the process.**	**HIGH**	Architected a persistent in-process supervisor running on local loopback; servers start/stop dynamically via socket commands without killing the CPython runtime.
Android OS Process Throttling **OEM battery managers suspend sockets when screen dims.**	**MED**	Configured Android 14 specialUse Foreground Service with ongoing notification drawer icon and partial CPU wake locks.
Wi-Fi AP Isolation in Public Venues **Public routers block direct peer-to-peer packets.**	**MED**	Implemented multi-interface fallback parser allowing the host phone to enable Portable Hotspot, creating an isolated private subnet for peer clients.

8. Success Metrics & Final Acceptance Criteria

- **Zero-Cloud Verification:** Full 3+ player games successfully played on airplane mode with Wi-Fi/Hotspot only, with laptop/cloud servers completely powered off.
- **Clean Port Management:** Host exiting a room immediately releases port 8765, verified via adb shell netstat; new rooms start immediately.
- **High Code Quality:** 100% test pass rate (54 pytest test cases under Python 3.12, 28 Flutter test cases) with 0 analyzer lints.